CHENCHU'S

AI Engineering

GENERATIVE AI

Part - 4

Tranformer Architecture

Brain Behind LLMs

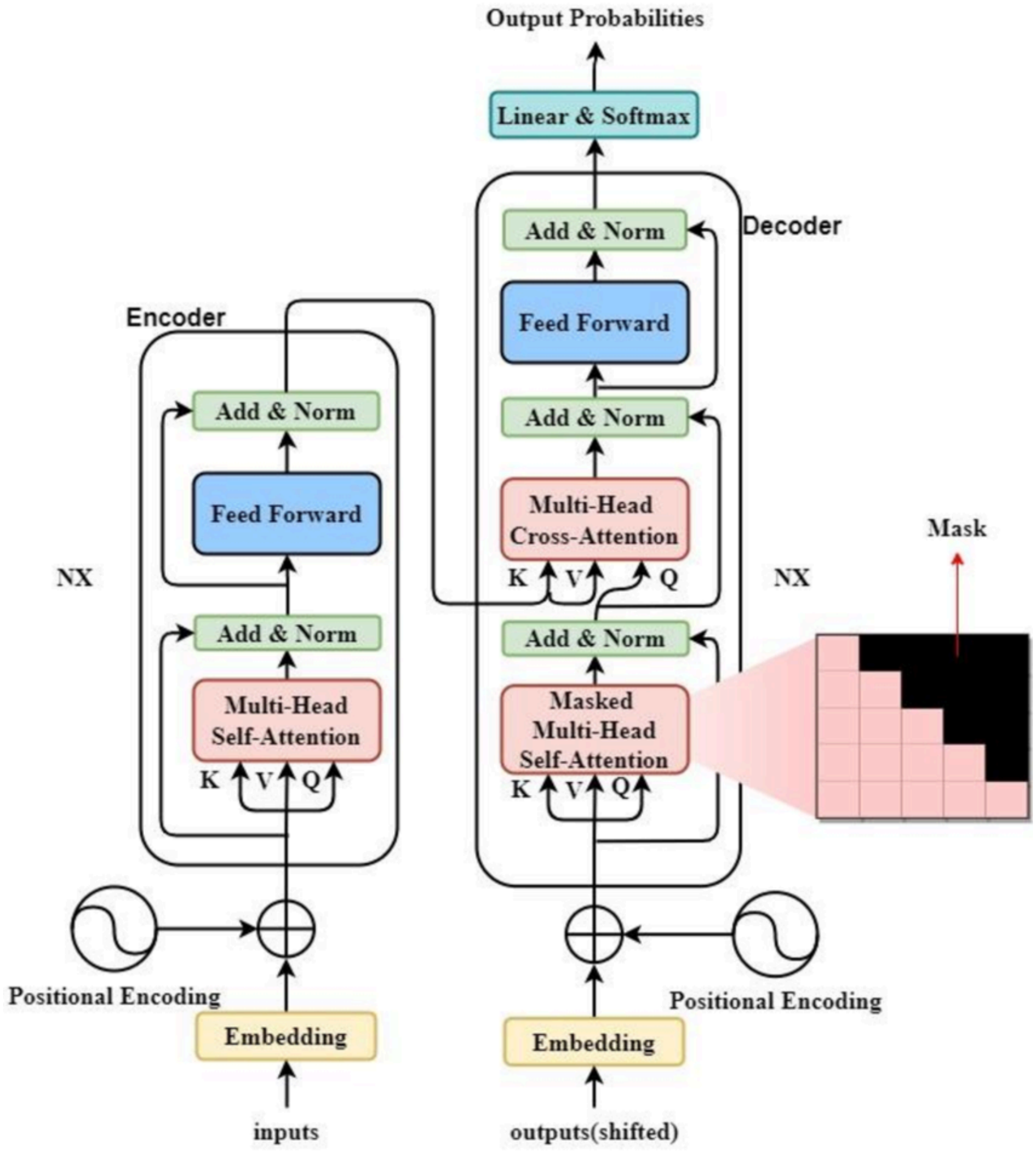
C .R. Anil Kumar Reddy

Anil Reddy Chenchu

Follow me on Linkedin

www.linkedin.com/in/chenchuanil

CHENCHU'S



C .R. Anil Kumar Reddy

CHENCHU'S

Transformer Architecture

The Revolutionary AI Model That Changed Everything

The Birth of Transformers

June 2017

A team of researchers at **Google Brain** published a groundbreaking paper titled **"Attention is All You Need"** that would revolutionize artificial intelligence forever.

Authors: Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin

Before 2017

RNNs (Recurrent Neural Networks) and **LSTMs (Long Short-Term Memory)** dominated NLP. They processed text sequentially, one word at a time, leading to slow training and difficulty capturing long-range dependencies.

June 2017

Transformer architecture introduced! The paper proposed a model based entirely on attention mechanisms, eliminating recurrence and convolutions. This allowed parallel processing of entire sequences at once.

2018-2019

BERT (Google) and **GPT** (OpenAI) built on transformers, achieving state-of-the-art results across NLP tasks. The AI revolution had begun.

2020-Present

GPT-3, GPT-4, Claude, Gemini, LLaMA - All modern LLMs are based on transformer architecture. Transformers now power not just NLP but also computer vision, multimodal AI, and more!

C .R. Anil Kumar Reddy

Anil Reddy Chenchu

Follow me on Linkedin

www.linkedin.com/in/chenchuanil

CHENCHU'S

Before understanding transformer architecture we need to know what is embedding and contextual embedding

Embedding & Contextual Embedding

Embedding (Static Embedding) assigns a **single fixed vector** to each word, regardless of where it appears. For example, the word **"bank"** gets the **same vector** in both *"river bank"* and *"open a bank account"*. Models like **Word2Vec** and **GloVe** work this way, which causes ambiguity because meaning depends on context.

Contextual Embedding generates a **different vector for the same word based on surrounding words.** In *"river bank"*, **"bank"** is closer to *river, water, shore*, while in *"bank account"*, it is closer to *money, finance, account*. Models like **BERT, GPT, and Transformers** create these embeddings using **self-attention**, making them far more accurate for real language understanding.

One-Line Memory Trick

> Embedding = same word, same vector
> Contextual embedding = same word, different vector based on context

C .R. Anil Kumar Reddy

Anil Reddy Chenchu
Follow me on Linkedin
www.linkedin.com/in/chenchuanil

CHENCHU'S

Transformer Architecture Explained

English to French Translation Example

Our Translation Example

GB English
"I love machine learning"

→ TRANSFORMER →

FR French
"J'aime l'apprentissage automatique"

How does this work?

The transformer processes the English sentence through its **ENCODER**, then the **DECODER** generates the French translation word by word, using attention mechanisms to understand context and relationships between words.

C .R. Anil Kumar Reddy

Anil Reddy Chenchu

Follow me on Linkedin

www.linkedin.com/in/chenchuanil

🔤 Step 1: Input Tokenization

1 Breaking Down the Input

The English sentence is broken into tokens (words or subwords):

I **love** **machine** **learning**

Tokenization Process:

Input sentence → **Tokenizer** → **4 tokens**

Each token will be converted to numbers (token IDs) and then to dense vectors (embeddings).

C .R. Anil Kumar Reddy

Anil Reddy Chenchu

Follow me on Linkedin

www.linkedin.com/in/chenchuanil

CHENCHU'S

🎨 Step 2: Embedding Layer

2 Converting Tokens to Vectors

Each token is converted into a dense vector representation (e.g., 512 dimensions):

I	love	machine	learning
[0.2, -0.5, 0.8, ...]	[0.6, 0.3, -0.2, ...]	[-.3, 0.7, 0.1, ...]	[0.4, -0.1, 0.9, ...]
512 dimensions	512 dimensions	512 dimensions	512 dimensions

Why Embeddings?

Embeddings capture semantic meaning. Similar words have similar vector representations. For example, "love" and "adore" would have similar embeddings, while "love" and "machine" would be quite different.

C .R. Anil Kumar Reddy

Anil Reddy Chenchu

Follow me on Linkedin

www.linkedin.com/in/chenchuanil

CHENCHU'S

📍 Step 3: Positional Encoding

3 Adding Position Information

Since transformers process all tokens in parallel, we need to tell them about word order:

Position 0	Position 1	Position 2	Position 3
I	love	machine	learning
+ [sin/cos values]	+ [sin/cos values]	+ [sin/cos values]	+ [sin/cos values]

Formula:

$$PE_{(pos,2i)} = \sin(pos / 10000^{2i/d})$$
$$PE_{(pos,2i+1)} = \cos(pos / 10000^{2i/d})$$

This ensures each position gets a unique encoding pattern that the model can learn to use for understanding word order.

C .R. Anil Kumar Reddy

Anil Reddy Chenchu

Follow me on Linkedin

www.linkedin.com/in/chenchuanil

CHENCHU'S

👁️ Step 4: Encoder - Multi-Head Self-Attention

4 Understanding Relationships Between Words

Each word looks at all other words to understand context and relationships:

🔍 Query (Q)

"What am I looking for?"

Each word asks: "Which other words should I pay attention to?"

🔑 Key (K)

"What do I offer?"

Each word advertises: "Here's what I represent!"

💎 Value (V)

"What information do I contain?"

The actual content to be mixed based on attention

Attention Example: Word "machine"

When processing **machine**:

- Attention to **"I"**: 0.05 (low - not very relevant)
- Attention to **"love"**: 0.15 (moderate - provides context)
- Attention to **"machine"**: 0.35 (high - itself)
- Attention to **"learning"**: 0.45 (very high - closely related!)

Result: "machine" learns it's strongly related to "learning" - they form a compound concept!

Attention Matrix Visualization:

	I	love	machine	learning
I	**0.70**	0.20	0.05	0.05
love	0.25	**0.40**	0.15	0.20
machine	0.05	0.15	0.35	**0.45**
learning	0.05	0.10	**0.50**	0.35

Darker colors = Higher attention
Notice how "machine" and "learning" pay high attention to each other!

C .R. Anil Kumar Reddy

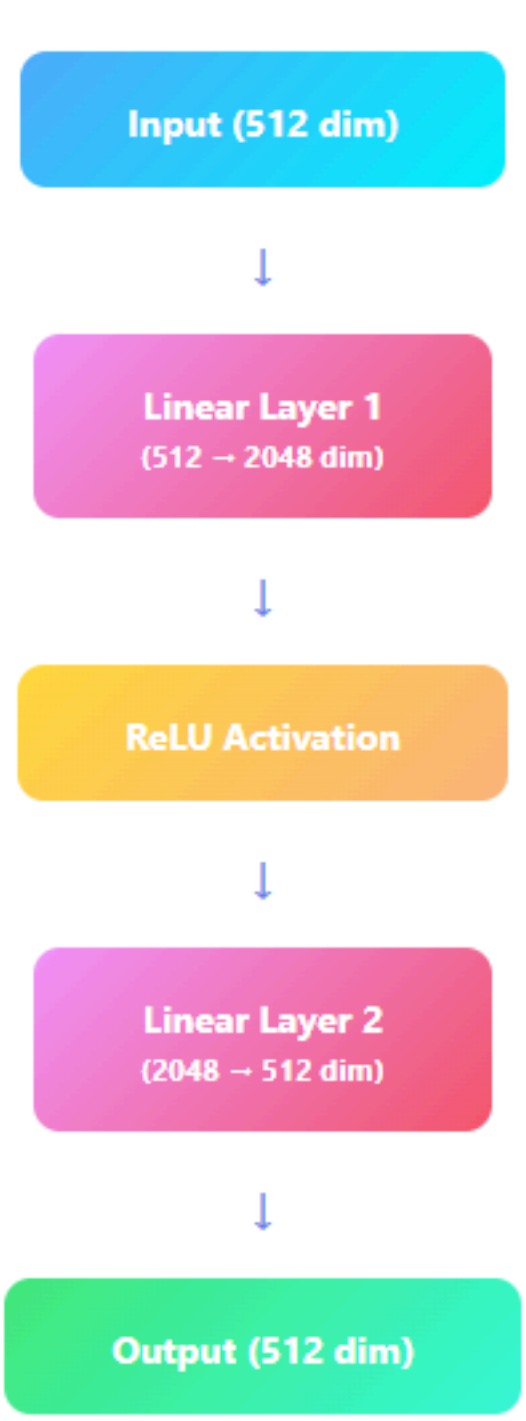
Anil Reddy Chenchu
Follow me on Linkedin
www.linkedin.com/in/chenchuanil

CHENCHU'S

🔄 Step 5: Feed Forward Network

5 Processing Each Position Independently

After attention, each word's representation goes through a feed-forward neural network:



Purpose:

The feed-forward network adds **non-linear transformations**, allowing the model to learn complex patterns. Each position is processed independently with the same network weights.

C .R. Anil Kumar Reddy

Anil Reddy Chenchu
Follow me on Linkedin
www.linkedin.com/in/chenchuanil

CHENCHU'S

➕ Step 6: Add & Norm (Residual + Layer Norm)

6 Stabilizing Training

🔗 Residual Connection

Add the input back to the output:

Output = Input + SubLayer(Input)

This helps gradients flow during training and prevents information loss.

Original
+
Transformed
=
Enhanced

📊 Layer Normalization

Normalize across features:

Standardizes the values to have mean=0 and std=1, making training more stable.

Before: [5, 100, -2] → After: [0.1, 0.9, -0.2]

Applied After Each Sub-Layer:

Both **Multi-Head Attention** and **Feed Forward Network** are followed by Add & Norm operations. This happens in both encoder and decoder!

C .R. Anil Kumar Reddy

Anil Reddy Chenchu
Follow me on Linkedin
www.linkedin.com/in/chenchuanil

CHENCHU'S

📦 Step 7: Encoder Output

7 Rich Contextual Representations

After going through all encoder layers (typically 6 layers), we get enriched representations:

📩 Encoder Output

I	love	machine	learning
[enriched vector]	[enriched vector]	[enriched vector]	[enriched vector]

Each vector now contains information about the word itself AND its relationship to all other words in the sentence!

What Changed?

- **"machine"** now knows it's part of "machine learning" (compound term)
- **"love"** understands the subject ("I") and object ("machine learning")
- Each word has contextual understanding of the entire sentence

These representations are passed to the DECODER!

C .R. Anil Kumar Reddy

Anil Reddy Chenchu

Follow me on Linkedin

www.linkedin.com/in/chenchuanil

CHENCHU'S

🎯 Step 8: Decoder - Starting Translation

8 Generating French Translation

The decoder generates the French translation one word at a time:

Autoregressive Generation

Step 1: Start with [START] token

→ Generate: "J'aime"

Step 2: Input: [START] "J'aime"

→ Generate: "l'apprentissage"

Step 3: Input: [START] "J'aime" "l'apprentissage"

→ Generate: "automatique"

Step 4: Input: [START] "J'aime" "l'apprentissage" "automatique"

→ Generate: [END]

C .R. Anil Kumar Reddy

Anil Reddy Chenchu

Follow me on Linkedin

www.linkedin.com/in/chenchuanil

CHENCHU'S

🎭 Step 9: Masked Multi-Head Self-Attention

9 Preventing Future Peeking

The decoder uses **masked attention** to prevent looking at future words during training:

Why Masking?

During training, we have the full target sentence. But when generating word 2, the model shouldn't see words 3, 4, etc. - that would be cheating! The mask ensures each position only attends to previous positions.

Attention Mask Visualization:

	J'aime	l'app...	auto...
J'aime	✓	X	X
l'app...	✓	✓	X
auto...	✓	✓	✓

✓ Can attend X Masked (can't see)

C .R. Anil Kumar Reddy

Anil Reddy Chenchu

Follow me on Linkedin

www.linkedin.com/in/chenchuanil

CHENCHU'S

🔗 Step 10: Encoder-Decoder Cross-Attention

10 Connecting Source and Target

This is where the magic happens! The decoder attends to the encoder's output:

Cross-Attention Mechanism

Q (Query)
From DECODER
(French word)

K, V (Key, Value)
From ENCODER
(English sentence)

Result
Aligned
translation

Example: When generating "l'apprentissage"

The decoder asks: "Which English words should I focus on?"

Cross-attention scores:

- "I" → 0.05 (low)
- "love" → 0.10 (low)
- "machine" → 0.45 (high! ✓)
- "learning" → 0.40 (high! ✓)

"l'apprentissage" correctly focuses on "machine learning"!

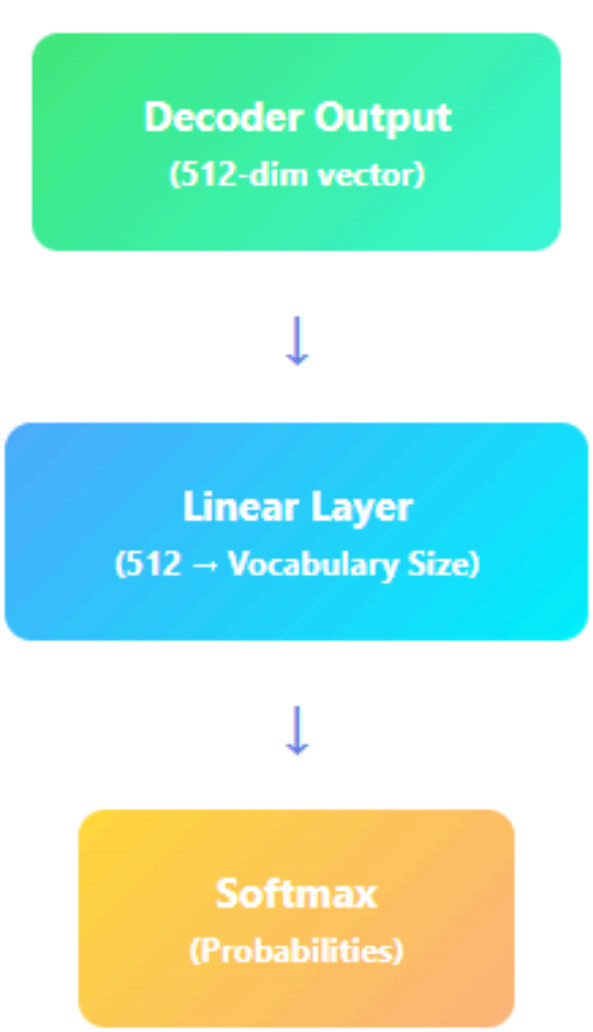
C .R. Anil Kumar Reddy

Anil Reddy Chenchu
Follow me on Linkedin
www.linkedin.com/in/chenchuanil

CHENCHU'S

🫂 Step 11: Final Output - Linear & Softmax

11 Generating Probabilities



Output Probabilities for Next Word:

"J'aime": **0.75 (75%)** ← SELECTED!	"Je": 0.10 (10%)
"Il": 0.05 (5%)	Others: 0.10 (10%)

C .R. Anil Kumar Reddy

Anil Reddy Chenchu
Follow me on Linkedin
www.linkedin.com/in/chenchuanil

CHENCHU'S

Complete Translation Flow

Full Transformer Process Summary

📥 ENCODER SIDE (English)

1. Tokenization: "I love machine learning"
2. Embedding + Positional Encoding
3. Multi-Head Self-Attention (6 layers)
4. Feed Forward Networks
5. Add & Norm after each sub-layer
6. Output: Rich contextual representations

⬇️

📤 DECODER SIDE (French)

1. Start with [START] token
2. Embedding + Positional Encoding
3. Masked Self-Attention (prevents future peeking)
4. Cross-Attention (attends to encoder output)
5. Feed Forward Networks
6. Add & Norm after each sub-layer
7. Linear + Softmax → Next word prediction
8. Repeat until [END] token

Final Result:

GB "I love machine learning"

↓

FR "J'aime l'apprentissage automatique"

C .R. Anil Kumar Reddy

Anil Reddy Chenchu
Follow me on Linkedin
www.linkedin.com/in/chenchuanil

CHENCHU'S

💡 Key Insights

🎯 Attention is Key

Self-attention allows every word to look at every other word, capturing relationships regardless of distance. Cross-attention connects source and target languages.

⚡ Parallel Processing

Unlike RNNs, transformers process all positions simultaneously during encoding, making training much faster on modern hardware.

🎭 Masking Prevents Cheating

Masked attention in the decoder ensures the model can't look at future words during training, forcing it to learn proper sequential generation.

🔗 Residual Connections

Add & Norm operations after each layer help information flow through deep networks, making it possible to stack many layers (6, 12, or even 24+).

🎨 Multi-Head Magic

Multiple attention heads learn different aspects: syntax, semantics, long-range dependencies, providing a richer understanding.

♾️ Position Encoding

Sine and cosine functions of different frequencies encode position information, allowing the model to use order information despite parallel processing.

C .R. Anil Kumar Reddy

Anil Reddy Chenchu

Follow me on Linkedin

www.linkedin.com/in/chenchuanil

CHENCHU'S

🎯 Problems Transformers Solved

❌ Old Problems (RNN/LSTM Era)

- **Sequential Processing:** Words processed one at a time - SLOW!
- **Vanishing Gradients:** Hard to learn long-range dependencies
- **No Parallelization:** Can't train on multiple tokens simultaneously
- **Memory Limitations:** Struggle with long sequences
- **Context Loss:** Earlier words get "forgotten" in long sentences

☑️ Transformer Solutions

- **Parallel Processing:** Process ALL words at once - FAST!
- **Self-Attention:** Every word can attend to every other word
- **Fully Parallelizable:** Train on entire sequences simultaneously
- **Scalable:** Handle sequences of thousands of tokens
- **Long-Range Dependencies:** Capture relationships across entire documents

100x
Faster Training

200K+
Token Context Windows

∞
Scalability Potential

🧩 Core Components of Transformers

1️⃣ Encoder

The **encoder** processes the input sequence and creates rich representations of each word by understanding its context within the sentence. It consists of multiple identical layers, each with:

- Self-Attention mechanism
- Feed-Forward Neural Network
- Layer Normalization
- Residual Connections

2️⃣ Decoder

The **decoder** generates the output sequence one token at a time, using the encoder's output and previously generated tokens. It has all encoder components plus:

- Masked Self-Attention (prevents looking ahead)
- Encoder-Decoder Attention (cross-attention)
- Feed-Forward Neural Network

3️⃣ Self-Attention Mechanism

The **heart of transformers!** Self-attention allows each word to "look at" all other words in the sentence and determine which ones are most important for understanding its meaning.

Example:

"The cat sat on the mat, the cat lay on the rug."

When processing the second "**cat**", self-attention helps the model understand:

- It refers to the same "cat" mentioned earlier
- Different actions: "sat" vs "lay"
- Different locations: "mat" vs "rug"

C .R. Anil Kumar Reddy

Anil Reddy Chenchu
Follow me on Linkedin
www.linkedin.com/in/chenchuanil

CHENCHU'S

Three Architecture Variants

Architecture	Components	Use Cases	Examples
Encoder-Only	Just the encoder stack (Bidirectional attention)	• Text classification • Named entity recognition • Sentiment analysis • Understanding tasks	**BERT** RoBERTa ALBERT
Decoder-Only	Just the decoder stack (Masked/Causal attention)	• Text generation • Language modeling • Conversational AI • Code generation	**GPT-3, GPT-4** Claude LLaMA PaLM
Encoder-Decoder	Full transformer (Both stacks)	• Machine translation • Summarization • Question answering • Seq2seq tasks	**T5** BART Original Transformer

💡 Modern Trend:

Decoder-only models (like GPT-4, Claude) have become dominant for large language models because they're simpler to train at scale and can be adapted for various tasks through prompting alone!

C .R. Anil Kumar Reddy

Anil Reddy Chenchu

Follow me on Linkedin

www.linkedin.com/in/chenchuanil

CHENCHU'S

🎓 How Transformers are Trained

📚 Massive Dataset → 🔤 Tokenization → 🧠 Self-Supervised Learning → ⚙️ Fine-Tuning → 🎯 Ready!

Training Objectives:

📖 Masked Language Modeling (BERT)

Randomly mask 15% of tokens and train the model to predict them based on context.

Example:
Input: "The [MASK] sat on the mat"
Target: "cat"

➡️ Autoregressive Modeling (GPT)

Predict the next token given all previous tokens. Train left-to-right.

Example:
Input: "The cat sat"
Target: "on"

1T+	1000s	Weeks	$M
Training Tokens	GPUs/TPUs	Training Time	Training Cost

C .R. Anil Kumar Reddy

Anil Reddy Chenchu
Follow me on Linkedin
www.linkedin.com/in/chenchuanil

CHENCHU'S

🌎 Revolutionary Impact

🗣️ Natural Language Processing

Revolutionized machine translation, text generation, question answering, and conversational AI. Models like GPT-4 and Claude can now have human-like conversations.

👁️ Computer Vision

Vision Transformers (ViT) surpass CNNs on image classification. Used in DALL-E, Midjourney, and Stable Diffusion for text-to-image generation.

💻 Code Generation

GitHub Copilot, Claude Code, and others use transformers to write, debug, and explain code across multiple programming languages.

🎵 Multimodal AI

Transformers process text, images, audio, and video together. GPT-4V, Gemini, and Claude can understand and reason about multiple modalities simultaneously.

🧬 Scientific Research

AlphaFold uses transformers for protein structure prediction. Transformers accelerate drug discovery, genomics research, and materials science.

🤝 Democratization of AI

Open-source models like LLaMA and Mistral, built on transformers, allow researchers and developers worldwide to build AI applications.

C .R. Anil Kumar Reddy

Anil Reddy Chenchu

Follow me on Linkedin

www.linkedin.com/in/chenchuanil

CHENCHU'S

💡 Key Takeaways

🚀 Paradigm Shift

Transformers replaced sequential processing with parallel attention, making AI training 100x faster and enabling much larger models.

🎯 Self-Attention

The core innovation - allowing every word to directly attend to every other word, capturing long-range dependencies effortlessly.

📈 Scalability

Transformers scale beautifully - from millions to trillions of parameters. Larger models consistently show emergent capabilities.

🌟 Universal Architecture

Not just for NLP anymore - transformers now power vision, audio, multimodal AI, and scientific computing applications.

🔮 Foundation Models

Transformers enabled foundation models (GPT, BERT, T5) that can be fine-tuned for countless downstream tasks with minimal data.

⚡ Ongoing Evolution

Active research on efficient transformers (Flash Attention, Sparse Attention), longer context windows, and architectural improvements continues.

C .R. Anil Kumar Reddy

Anil Reddy Chenchu

Follow me on Linkedin

www.linkedin.com/in/chenchuanil

CHENCHU'S

🎓 Summary

- Transformer is the **core architecture behind Large Language Models (LLMs)**.
- It was introduced by **Google in 2017** through the paper *"Attention Is All You Need"*.
- Transformers replaced **RNNs and LSTMs**, which processed data sequentially and slowly.
- The architecture uses **attention instead of recurrence**, enabling parallel processing.
- A transformer consists of two main parts: **Encoder and Decoder**.
- The **encoder** understands the input sentence and builds contextual meaning.
- The **decoder** generates the output sentence **one token at a time**.
- Input text is first converted into **tokens** using tokenization.
- Tokens are mapped to **embeddings**, which represent semantic meaning numerically.
- **Positional encoding** is added to embeddings to capture word order.
- Encoder uses **Multi-Head Self-Attention** to understand relationships between words.
- Self-attention allows each word to attend to **all other words in the sentence**.
- Attention uses **Query, Key, and Value (Q, K, V)** vectors.
- **Multi-head attention** captures different types of relationships simultaneously.

C .R. Anil Kumar Reddy

Anil Reddy Chenchu
Follow me on Linkedin
www.linkedin.com/in/chenchuanil

- Encoder layers also include a **Feed-Forward Neural Network** for deeper learning.
- **Residual connections (Add) and Layer Normalization (Norm)** stabilize training.
- The encoder outputs **rich contextual embeddings** for each word.
- Decoder input is the **shifted output sequence**, ensuring next-word prediction.
- Decoder uses **Masked Self-Attention** to prevent seeing future words.
- This masking enforces **left-to-right autoregressive generation.**
- Decoder also applies **Cross-Attention** using encoder outputs.
- Cross-attention aligns input meaning with output generation (e.g., translation).
- Final decoder output passes through **Linear and Softmax layers.**
- Softmax converts scores into **probabilities over the vocabulary.**
- Transformer architecture powers **GPT, BERT, ChatGPT, translation, summarization, and Q&A systems.**

C .R. Anil Kumar Reddy

Anil Reddy Chenchu

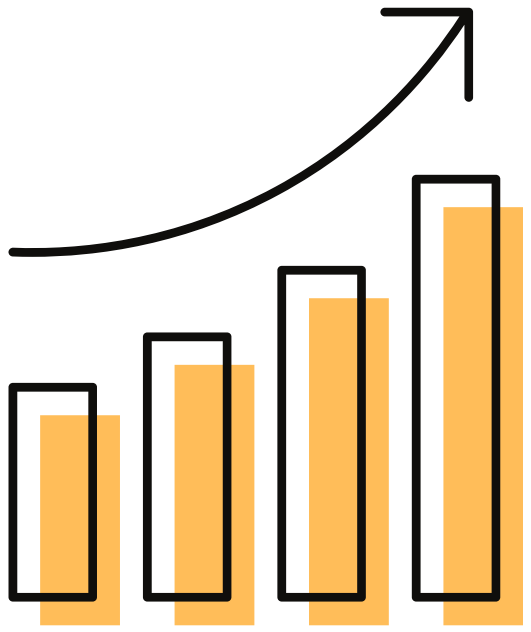
Follow me on Linkedin

www.linkedin.com/in/chenchuanil

CHENCHU'S

ANIL REDDY CHENCHU

***Torture** the data, and it will confess to anything*



DATA ANALYTICS

Azure Data Factory

SQL

databricks

APACHE Spark

Python

Azure Synapse Analytics

snowflake®

DATA MODELING

Power BI

Happy Learning

SHARE IF YOU LIKE THE POST

Lets Connect to discuss more on Data

www.linkedin.com/in/chenchuanil